Universitat d'Alacant
Universidad de Alicante

Tecnología y arquitectura robótica (34036)
Práctica 1

David Sanchez Mas 74394689Q

Febrero 2026

Índice

1 Preguntas **3**

2 Ejercicios **9**

2.1 Primeros pasos 9

2.2 Nivel principiante 10

2.3 Nivel intermedio 12

1 Preguntas

1. **¿Qué podemos saber de este nodo? (Suscriptores, topics)**
   Si ejecutamos el comando *ros2 node info /talker*, podemos ver que el nodo está suscrito a */parameter_events* y que publica en tres topics distintos: */chatter*, */parameter_events* y */rosout*.

```
davidetsm-docker@david:/workspace$ ros2 node info /talker
/talker
  Subscribers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
  Publishers:
    /chatter: std_msgs/msg/String
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /talker/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /talker/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /talker/get_parameters: rcl_interfaces/srv/GetParameters
    /talker/list_parameters: rcl_interfaces/srv/ListParameters
    /talker/set_parameters: rcl_interfaces/srv/SetParameters
    /talker/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```

Figure 1: Información del nodo /talker

2. **¿Qué podemos saber sobre el topic que publica el nodo al que acabamos de lanzar?**
   Si ejecutamos el comando *ros2 topic info /chatter*, podemos ver que es de tipo *std_msgs/msg/String*, que tiene un publicador y ninguno nodo suscrito.

```
davidetsm-docker@david:/workspace$ ros2 topic info /chatter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 0
```

Figure 2: Información del topic /chatter

3. **¿Qué podemos saber sobre este nuevo nodo?**
   El nuevo nodo es un suscriptor del topic */chatter* y su función es recibir los mensajes que publica el primer nodo.

```
davidetsm-docker@david:/workspace$ ros2 node info /listener
/listener
  Subscribers:
    /chatter: std_msgs/msg/String
  Publishers:
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
  Service Servers:
    /listener/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /listener/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /listener/get_parameters: rcl_interfaces/srv/GetParameters
    /listener/list_parameters: rcl_interfaces/srv/ListParameters
    /listener/set_parameters: rcl_interfaces/srv/SetParameters
    /listener/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
  Service Clients:

  Action Servers:

  Action Clients:
```

Figure 3: Información del nodo /listener

4. **¿Qué ha sucedido ahora en el topic /chatter?**
   Ahora en el topic tenemos un suscriptor, es decir, el mensaje del /talker se recibe por el /listener correctamente.

```
davidetsm-docker@david:/workspace$ ros2 topic info /chatter
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
```

Figure 4: Información del topic /chatter

5. **¿Qué podemos saber a partir de este gráfico?**
   Cuando ejecutamos la herramienta, obtenemos el siguiente gráfico:

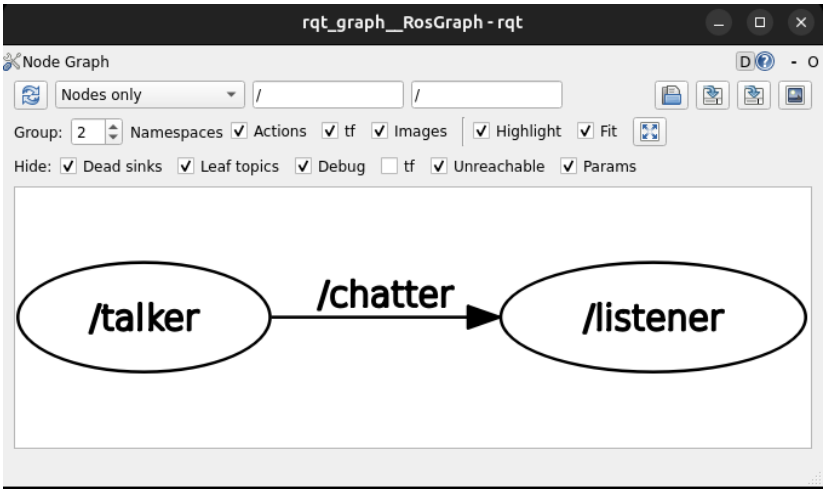


Figure 5: Nodos y topics de la primera prueba

Podemos ver que la comunicación se ha establecido entre los nodos */talker* y */listener* mediante el topic */chatter*.

6. **¿Qué puedes comentar acerca de este servicio?**
   Es un servicio cliente-servidor, donde el cliente manda los dos enteros que han sido introducidos por parámetros en su ejecución, el servidor calcula la suma de los dos y lo vuelve a mandar.

   El servidor crea el nodo */minimal_service* y el cliente */minimal_client_async*

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run servicio_suma client 3 7
[INFO] [1771406144.390233100] [minimal_client_async]: Result of add_two_ints: fo
r 3 + 7 = 10
```

Figure 6: Ejecución del servicio suma

7. **Al lanzar el servidor y a continuación ejecutar el comando** *ros2 service list* **?¿Qué servicios aparecen?¿Cómo se ha creado el servicio** *minimal_service* **y cuál es su cometido?**
   Podemos ver que se han creado: */add_two_ints*, y */minimal_service* con todas las opciones para recoger, listar y modificar parámetros. El servicio minimal_service se crea en el código del servidor con la instrucción: *super().__init__('minimal_service')*

8. **¿Cómo puedo saber cuáles son los parámetros de la "petición" y de la "respuesta" de un servicio? ¿De qué tipología son estos parámetros? (Pista: Muéstralo con el comando** *ros2 interface show* **).**
   Ejecutamos el comando *ros2 interface show example_interfaces/srv/AddTwoInts* y vemos la siguiente salida:

```
davidetsm-docker@david:/workspace/ros2_ws/src$ ros2 interface show interfaz_serv
icio_suma/srv/AddTwoInts
int64 a
int64 b
---
int64 sum
```

Figure 7: Parámetros de petición y respuesta

   De aquí podemos saber que los parámetros de la petición son dos enteros de 64 bits (a y b) y los de la respuesta es otro entero de 64 bits (sum).

9. **Habitualmente las dependencias que necesitarán nuestros paquetes para funcionar son las vistas anteriormente, sin embargo, se pueden añadir otras ¿Qué comando me indica todas las dependencias que puedo usar en mis paquetes?**
   Si utilizamos el comando *ros2 pkg list* podemos ver la lista de dependencias:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 pkg list
ackermann_msgs
ackermann_steering_controller
action_msgs
action_tutorials_cpp
action_tutorials_interfaces
action_tutorials_py
actionlib_msgs
actuator_msgs
admittance_controller
ament_cmake
ament_cmake_auto
ament_cmake_copyright
ament_cmake_core
ament_cmake_cppcheck
ament_cmake_cpplint
ament_cmake_export_definitions
ament_cmake_export_dependencies
ament_cmake_export_include_directories
ament_cmake_export_interfaces
ament_cmake_export_libraries
ament_cmake_export_link_flags
ament_cmake_export_targets
```

Figure 8: Lista de dependencias disponibles

10. **¿Qué comando se debe utilizar?**
    Si utilizamos el comando *ros2 interface show interfaz/msg/MiMensaje* desde el directorio src:

```
davidetsm-docker@david:/workspace/ros2_ws/src$ ros2 interface show interfaz/msg/MiMensaje
int32 x
int32 y
string NOMBRE=David
```

Figure 9: Parámetros del mensaje

11. **Muestra el gráfico de entorno ROS generado, usando rqt_graph.**
    Si ejecutamos cliente y servidor, podemos ver:

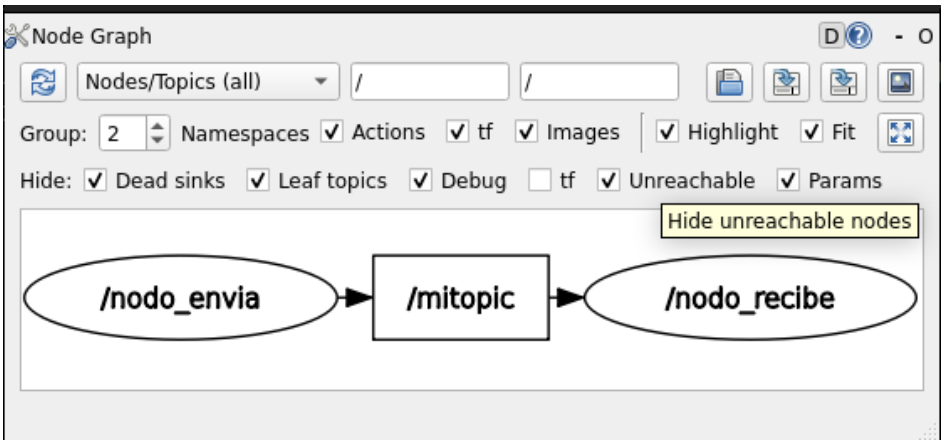


Figure 10: Nodos y topic del primer paquete

12. **Intenta modificar la constante nombre, del mensaje, modificando el código del nodo** *nodo_envia* **.¿Qué sucede?**
    Si intentamos modificar la variable *msg.NOMBRE* e intentamos ejecutar, nos da el siguiente error:

```
 line 20, in timer_callback
    msg.NOMBRE = "Daniel"
AttributeError: 'MiMensaje' object attribute 'NOMBRE' is read-only
[ros2run]: Process exited with failure 1
```

Figure 11: Error al intentar modificar el parámetro NOMBRE

13. **¿De qué tipo es el topic que aparece?**
    Si con servidor y cliente en funcionamiento ejecutamos *ros2 topic info /mitopic* podemos ver la siguiente salida:

```
davidetsm-docker@david:/workspace$ ros2 topic info /mitopic
Type: interfaz/msg/MiMensaje
Publisher count: 1
Subscription count: 1
```

Figure 12: Información del topic /mitopic

Podemos ver que es de tipo MiMensaje, el que hemos creado en la interfaz al principio de la sección.

14. **¿Con que nombre se han lanzado los nodos?**
    Si ejecutamos el archivo launch, podemos ver que los nodos se lanzan con los nombres *nodo_envia* y *nodo_recibe*:

```
davidetsm-docker@david:/workspace/ros2_ws/src/primer_paquete/launch$ ros2 launch
 mi_launch.py
[INFO] [launch]: All log files can be found below /home/davidetsm-docker/.ros/lo
g/2026-02-20-16-01-16-561429-david-5985
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [server-1]: process started with pid [5986]
[INFO] [client-2]: process started with pid [5988]
[server-1] [INFO] [1771599677.286674214] [nodo_envia]: Enviando MiMensaje: Nombr
e=David x=0, y=0
[client-2] [INFO] [1771599677.286683592] [nodo_recibe]: Recibido MiMensaje: Nomb
re=David x=0, y=0
[server-1] [INFO] [1771599677.777076929] [nodo_envia]: Enviando MiMensaje: Nombr
e=David x=1, y=2
```

Figure 13: Nodos del ejercicio launch

15. **¿Cómo podemos saber que se ha lanzado la acción correctamente? ¿Qué método me permite saber que la acción ha finalizado exitosamente?**
    Si lanzamos el servidor y ejecutamos el comando *ros2 action list* podemos ver las acciones que se están ejecutando:

```
davidetsm-docker@david:/workspace$ ros2 action list
/fibonacci
```

Figure 14: Acciones activas

Podemos saber si la acción ha terminado porque el servidor tiene el método **goal_handle.succeed()** que nos indica que el proceso ha terminado.

16. **Sin crear un nodo, ¿cómo lanzarías la acción Fibonnaci? ¿Cómo observarías el resultado de esta acción?**
    Para poder lanzar la acción sin crear un nodo desde la terminal, todo con el servidor corriendo, podemos ejecutar el comando:

ros2 action send_goal /fibonacci interfaz/action/Fibonacci "{orden: 10}"

Entonces, podemos ver el resultado:

```
davidetsm-docker@david:/workspace/ros2_ws/src$ ros2 action send_goal /fibonacci interfaz/action
/Fibonacci "{orden: 10}" --feedback
Waiting for an action server to become available...
Sending goal:
     orden: 10

Feedback:
    secuencia_actual:
- 0
- 1
- 1
- 2
```

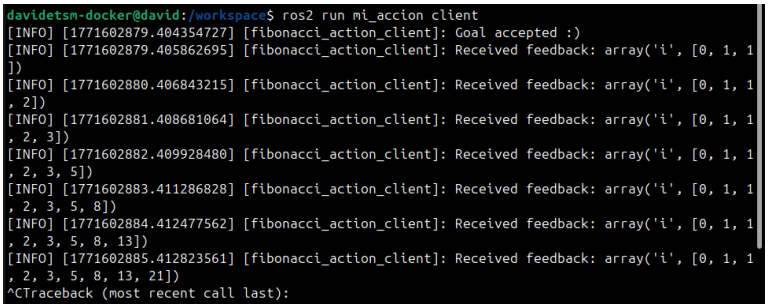
Figure 15: Resultado de Fibonacci

Podemos comprobar que ha funcionado porque al final de la ejecución tenemos un mensaje:

Goal finished with status: SUCCEED

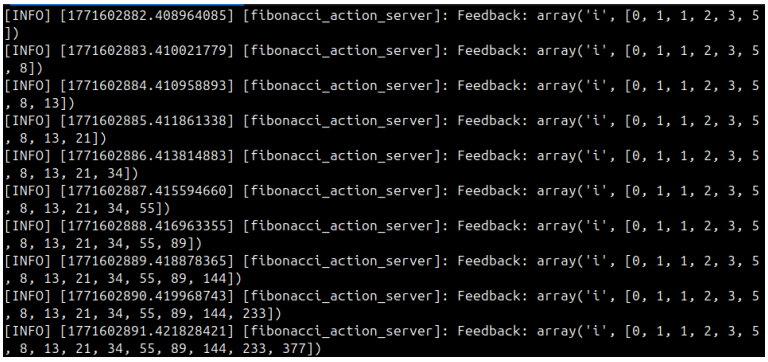
17. **Si paramos el nodo a mitad de la ejecución de la acción, ¿se cancela la acción? ¿Se puede cancelar la acción desde la terminal?**

Como podemos ver en las siguientes imágenes, la ejecución del servidor no para aunque se cancele la ejecución del nodo cliente con *Ctrl + C*:



```
davidetsm-docker@david:/workspace$ ros2 run mi_accion client
[INFO] [1771602879.404354727] [fibonacci_action_client]: Goal accepted :)
[INFO] [1771602879.405862695] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
])
[INFO] [1771602880.406843215] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2])
[INFO] [1771602881.408681064] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2, 3])
[INFO] [1771602882.409928480] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2, 3, 5])
[INFO] [1771602883.411286828] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2, 3, 5, 8])
[INFO] [1771602884.412477562] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2, 3, 5, 8, 13])
[INFO] [1771602885.412823561] [fibonacci_action_client]: Received feedback: array('i', [0, 1, 1
, 2, 3, 5, 8, 13, 21])
^CTraceback (most recent call last):
```

Figure 16: Parada de ejecución del cliente



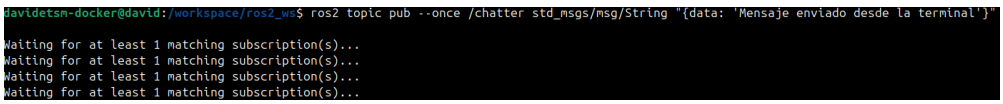
```
[INFO] [1771602882.408964085] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
])
[INFO] [1771602883.410021779] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8])
[INFO] [1771602884.410958893] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13])
[INFO] [1771602885.411861338] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21])
[INFO] [1771602886.413814883] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34])
[INFO] [1771602887.415594660] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34, 55])
[INFO] [1771602888.416963355] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34, 55, 89])
[INFO] [1771602889.418878365] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34, 55, 89, 144])
[INFO] [1771602890.419968743] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34, 55, 89, 144, 233])
[INFO] [1771602891.421828421] [fibonacci_action_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5
, 8, 13, 21, 34, 55, 89, 144, 233, 377])
```

Figure 17: Parada de ejecución del servidor

2 Ejercicios

2.1 Primeros pasos

1. **Publica desde la terminal un mensaje, con la cadena de caracteres que desees al topic /chatter, para que este sea leído por el nodo /listener. Escribe los comandos que has tenido que ejecutar para que esto suceda. (Pista: Usa el comando ros2 topic pub)**
   Para poder mandar un mensaje desde la terminal, utilizamos el siguiente comando:
   ros2 topic pub –once /chatter std_msgs/msg/String "{data: 'Mensaje enviado desde la terminal'}"



```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 topic pub --once /chatter std_msgs/msg/String "{data: 'Mensaje enviado desde la terminal'}"

Waiting for at least 1 matching subscription(s)...
Waiting for at least 1 matching subscription(s)...
Waiting for at least 1 matching subscription(s)...
Waiting for at least 1 matching subscription(s)...
```

Figure 18: Mensaje publicado desde la terminal

2. **Cambia el servicio servicio_suma para que haga la siguiente operación: a + b - c * d / e, siendo a, b, c, d, e valores enteros (int64) pasados por línea de comandos, el**

resultado debe estar en formato float64. Describe que has tenido que hacer y los archivos que has tenido que modificar.
En primer lugar, modificamos la interfaz para ampliar el número de parámetros que puede recibir y procesar nuestro servidor:

```
davidetsm-docker@david:/workspace/ros2_ws/src$ ros2 interface show interfaz_serv
icio_suma/srv/AddTwoInts
int64 a
int64 b
int64 c
int64 d
int64 e
---
float64 result
```

Figure 19: Nuevos parámetros de servicio_suma

Después, modificamos los códigos de servidor y cliente para que cambien las operaciones. Una vez ejecutamos:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run servicio_suma client 10 8 6 4 2
[INFO] [1771408188.425857878] [minimal_client_async]: Result of operation: for 10 + 8 - 6 * 4 / 2 = 6
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run servicio_suma client 2 4 6 8 10
[INFO] [1771408206.300578878] [minimal_client_async]: Result of operation: for 2 + 4 - 6 * 8 / 10 = 1
```

Figure 20: Resultado de la modificación del servicio

2.2 Nivel principiante

1. **Crea un paquete nuevo que se llame p2pkg. A continuación, en el paquete interfaz define un mensaje llamado P2pkg_mensaje.msg con los siguientes campos**
Con el comando:
ros2 pkg create –build-type ament_python –license Apache-2.0 p2pkg –dependencies rclpy std_msgs, podemos crear el paquete p2pkg. A continuación, creamos el nuevo mensaje con el comando:
touch src/interfaz/msg/P2pkg_mensaje.msg
y modificando su contenido para poner el siguiente contenido:

```
P2pkg_mensaje.msg ×
ros2_ws > src > interfaz > msg > P2pkg_mensaje.msg
1  int32 numero
2  geometry_msgs/Pose posicion
3  string fecha
```

Figure 21: Parámetros del mensaje nuevo

Además, debemos añadir en el archivo CMakeLists.txt del paquete interfaz la condición de que se compile el mensaje:

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/MiMensaje.msg"
  "msg/P2pkg_mensaje.msg"
  DEPENDENCIES std_msgs
)
```

Figure 22: Añadir nuevo mensaje al CMakeLists.txt

2. **En el paquete p2pkg, crea un nodo publisher, llamado nodopub_ejercicio2, y un nodo subscriber, llamado nodosub_ejercicio2, que sean capaces de enviar y recibir el tipo de mensajes a través un topic al que llamaremos /topic_ejercicio2. Estos nodos deben ser creados dentro del paquete ROS p2pkg.** Dentro de la carpeta p2pkg/p2pkg creamos tanto el nodo publisher como el nodo subscriber y los añadimos al archivo setup.py como servidor y cliente, respectivamente:

```
entry_points={
    'console_scripts': [
        'server = p2pkg.nodopub_ejercicio2:main',
        'client = p2pkg.nodosub_ejercicio2:main'
    ],
},
```

Figure 23: Configuración de cliente y servidor

Ahora, en el paquete interfaz añadimos las dependencias de *geometry_msgs* en CMakeLists.txt y en package.xml:

```
find_package(std_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(geometry_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/MiMensaje.msg"
  "msg/P2pkgMensaje.msg"
  DEPENDENCIES std_msgs geometry_msgs
)
```

Figure 24: Dependencias de *geometry_msgs* en CMakeLists.txt

```
<depend>std_msgs</depend>
<build_depend>geometry_msgs</build_depend>
<exec_depend>geometry_msgs</exec_depend>
```

Figure 25: Dependencias de *geometry_msgs* en package.xml

Solo nos queda completar el código del publisher y el subscriber y lanzarlos:

```
[INFO] [1771584770.194361311] [nodopub_ejercicio2]: Enviando mensaje: numero=0 x
=0.0, w=1.0, fecha=20/02/2026
[INFO] [1771584770.680373430] [nodopub_ejercicio2]: Enviando mensaje: numero=1 x
=0.3110387263523706, w=0.6968447142882334, fecha=20/02/2026
[INFO] [1771584771.180363505] [nodopub_ejercicio2]: Enviando mensaje: numero=2 x
=0.651567536508992, w=0.8398228153832474, fecha=20/02/2026
[INFO] [1771584771.680343928] [nodopub_ejercicio2]: Enviando mensaje: numero=3 x
=0.501740153343283, w=0.5798746072025514, fecha=20/02/2026
```

Figure 26: Ejecución del ejercicio 4 de parte del publicador

```
[INFO] [1771584947.235741465] [nodosub_ejercicio2]: Recibido mensaje: numero=35
x=0.09449754080969708, w=0.07259704536281852, fecha=20/02/2026
[INFO] [1771584947.713027412] [nodosub_ejercicio2]: Recibido mensaje: numero=36
x=0.3783366378340093, w=0.446376533619646, fecha=20/02/2026
[INFO] [1771584948.213163914] [nodosub_ejercicio2]: Recibido mensaje: numero=37
x=0.40250149184262063, w=0.12601115383696326, fecha=20/02/2026
[INFO] [1771584948.713099882] [nodosub_ejercicio2]: Recibido mensaje: numero=38
x=0.130978814074027, w=0.513272710320234, fecha=20/02/2026
[INFO] [1771584949.213193572] [nodosub_ejercicio2]: Recibido mensaje: numero=39
x=0.9530711347877101, w=0.19679719185328892, fecha=20/02/2026
```

Figure 27: Ejecución del ejercicio 4 de parte del suscript

2.3 Nivel intermedio

1. **Genera un fichero llamado launch_ejercicio3.py que sea capaz de lanzar los nodos creados en el ejercicio anterior al mismo tiempo. El argumento de entrada se debe añadir por línea de comandos y tener un valor por defecto de 7 (Tienes que ver cómo hacer que el valor pasado por línea de comandos se guarde como argumento en el launch).**
   Creamos la carpeta *launch* en el paquete *mi_accion* y creamos en ella el archivo launch. Para poder incluir argumentos, debemos importar la librería *DeclareLaunchArgument* y configurarlo de la siguiente manera:

```
numero_argumento = DeclareLaunchArgument(
    'numero_argumento',
    default_value='7',
    description='Número entero de entrada para Fibonacci'
)
```

Figure 28: Configuración del DeclareLaunchArgument

De la siguiente manera, podemos configurar los nodos publicador y suscriptor:

```python
# Definimos el nodo publicador con el argumento
nodo_publicador = Node(
    package='mi_accion',
    executable='server',
    name='fibonacci_server',
    parameters=[{
        'numero': LaunchConfiguration('numero_argumento')
    }]
)

# Definimos el nodo suscriptor
nodo_suscriptor = Node(
    package='mi_accion',
    executable='client',
    name='fibonacci_client'
)
```

Figure 29: Configuración del nodo publicador y suscriptor

Ahora, para ejecutar el launch de nuestro archivo, ejecutamos el siguiente comando:

ros2 launch launch_ejercicio3.py numero_argumento:=10

```
[INFO] [server-1]: process started with pid [1823]
[INFO] [client-2]: process started with pid [1825]
[server-1] [INFO] [1772010094.605203038] [fibonacci_server]: Executing goal...
[client-2] [INFO] [1772010094.606443079] [fibonacci_client]: Goal accepted :)
[server-1] [INFO] [1772010094.606500257] [fibonacci_server]: Feedback: array('i', [0, 1, 1])
[client-2] [INFO] [1772010094.609860924] [fibonacci_client]: Received feedback: array('i', [0, 1, 1])
[server-1] [INFO] [1772010095.609152380] [fibonacci_server]: Feedback: array('i', [0, 1, 1, 2])
[client-2] [INFO] [1772010095.611253331] [fibonacci_client]: Received feedback: array('i', [0, 1, 1, 2])
[server-1] [INFO] [1772010096.611956724] [fibonacci_server]: Feedback: array('i', [0, 1, 1, 2, 3])
[client-2] [INFO] [1772010096.614741131] [fibonacci_client]: Received feedback: array('i', [0, 1, 1, 2, 3])
[server-1] [INFO] [1772010097.614207886] [fibonacci_server]: Feedback: array('i', [0, 1, 1, 2, 3, 5])
[client-2] [INFO] [1772010097.616582840] [fibonacci_client]: Received feedback: array('i', [0, 1, 1, 2, 3, 5])
```

Figure 30: Ejecución de los nodos de Fibonacci con el archivo launch

2. **Crea otro fichero llamado launch_ejercicio4.py que agrupe estos dos nodos dentro de un mismo grupo, llamado miGrupo. Haz uso de la etiqueta remapping para que el topic también se encuentre dentro de ese grupo. (Pista: namespace, remapping)** Seguimos la misma estructura que el archivo del ejercicio anterior pero, ahora debemos usar los namespaces para que servidor y cliente se muevan dentro del mismo grupo. También, pasamos el topic dentro del grupo. La configuración quedaría de la siguiente manera:

```python
# Definimos el nombre del grupo para luego añadirlo al namespace
nombre_grupo = 'miGrupo'

# Definimos el nodo publicador con el argumento
nodo_publicador = Node(
    package='mi_accion',
    executable='server',
    namespace=nombre_grupo,
    name='publicador_ejercicio4',
    parameters=[{
        'numero': LaunchConfiguration('numero_argumento')
    }],
)

# Definimos el nodo suscriptor
nodo_suscriptor = Node(
    package='mi_accion',
    executable='client',
    namespace=nombre_grupo,
    name='suscriptor_ejercicio4',
)
```

Figure 31: Configuración del grupo

Si lanzamos el archivo launch, podemos comprobar los nodos y los topics:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 action list
/miGrupo/fibonacci
davidetsm-docker@david:/workspace/ros2_ws$ ros2 node list
/miGrupo/publicador_ejercicio4
/miGrupo/suscriptor_ejercicio4
```

Figure 32: Comprobación del grupo

3. **Genera una nueva acción, ejFibonacci.action donde el goal y el result sean el mismo que Fibonacci.action, pero el feedback sea de tipo float64.**
   Dentro de la carpeta *action* creamos el archivo *ejFibonacci.action*

```
int32 orden # Objetivo
---
int32[] secuencia_final # Resultado
---
float64 raiz # Feedback
```

Figure 33: Configuración del nuevo archivo .action

```
davidetsm-docker@david:/workspace/ros2_ws/src$ ros2 interface show interfaz/action/EjFibonacci
int32 orden # Objetivo
---
int32[] secuencia_final # Resultado
---
float64 raiz # Feedback
```

Figure 34: Configuración de la nueva interfaz

4. **Crea un fichero ejercicios_fibServer.py que sea el servidor de la acción ejFibonacci para que se comporte igual que fibonacci_action_server.py, con la diferencia de que el feedback ha de ser la raiz cuadrada de la media de la secuencia de Fibonacci que se está calculando en cada iteración.**
   Cogiendo el código de ejemplo, podemos modificar el código para que vaya calculando la media en cada paso y poder devolver la raíz cuadrada:

```python
# Procesamos tantas veces como pida el cliente
for i in range(1, goal_handle.request.orden):
    nuevo_num = secuencia[i] + secuencia[i-1]
    secuencia.append(nuevo_num)

    # Calculamos la media y la raiz cuadrada
    media = sum(secuencia) / len(secuencia)
    feedback_msg.raiz_media_secuencia = math.sqrt(media)

    self.get_logger().info(f'Feedback (Raíz Media): {feedback_msg.raiz_media_secuencia:.4f}')
    goal_handle.publish_feedback(feedback_msg)
```

Figure 35: Procedimiento para calcular la raíz de la media en cada paso

5. **Crea un fichero ejercicios_fibClient.py que sea el nodo cliente de la acción, para que el orden de la secuencia sea modificable a través de un parámetro de entrada. Este nodo cliente publica la frase 'en proceso' a un topic llamado /estado_accion mientras la acción esté en proceso. Finalmente, debe mostrar el resultado obtenido de la acción por pantalla.** Basándonos en el código del cliente y modificándolo para que publique la frase "en_proceso", podemos ejecutar y obtener el siguiente output:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run mi_accion ej_fibClient
[INFO] [1772016497.054048142] [ejercicio_fibClient]: Esperando al servidor... Objetivo: 7
[INFO] [1772016497.058552543] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 0.8165
[INFO] [1772016498.062244272] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 1.0000
[INFO] [1772016499.064886831] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 1.1832
[INFO] [1772016500.067726654] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 1.4142
[INFO] [1772016501.070574842] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 1.6903
[INFO] [1772016502.073369467] [ejercicio_fibClient]: Feedback recibido (Raíz Media): 2.0310
[INFO] [1772016503.077014669] [ejercicio_fibClient]: Acción finalizada. Resultado: array('i', [0, 1, 1, 2, 3, 5, 8, 13])
```

Figure 36: Resultado de la ejecución con la raíz cuadrada de la media

Podemos ver también como se va publicando el mensaje:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 topic echo /estado_accion
data: en proceso
---
data: en proceso
---
data: en proceso
---
data: en proceso
---
data: en proceso
---
data: en proceso
---
```

Figure 37: Publicación del mensaje "en_proceso"

6. **Crea un paquete llamado service_temp, este paquete deberá albergar un servicio de ROS 2. Este servicio se encargará de realizar conversiones de temperatura, concretamente de Grados Celcius a Farenheit y viceversa. El funcionamiento es el siguiente:**

   - **Servidor: Recibe el valor de temperatura y realiza la conversión.**
   - **Cliente: Envía el valor de temperatura y el tipo de conversión a realizar y muestra el resultado obtenido desde el servidor. Ten en cuenta que el valor de temperatura deberá especificarse desde la línea de comandos del terminal al lanzar el nodo correspondiente.**

   Creamos el paquete *service_temp* y dentro de la interfaz *interfaz* creamos la carpeta *srv* donde poder definir la interfaz de servicio. Después de crear los archivos de servidor y cliente e incluirlos en el archivo *setup.py*, podemos realizar la ejecución:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run service_temp server
[INFO] [1772019280.458141716] [temp_server]: Servidor de conversión de temperatura
[INFO] [1772019282.574652360] [temp_server]: Petición: 100.0 (Celsius a Fahrenheit) -> Resultado: 212.0
[INFO] [1772019295.836259092] [temp_server]: Petición: 212.0 (Fahrenheit a Celsius) -> Resultado: 100.0
```

Figure 38: Ejecución de parte del servidor

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run service_temp client 100 Cel_to_Far
[INFO] [1772019282.588454320] [temp_client]: Resultado de la conversión: 212.0
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run service_temp client 212.0 Far_to_Cel
[INFO] [1772019295.846891039] [temp_client]: Resultado de la conversión: 100.0
```

Figure 39: Ejecución de parte del cliente

7. **Implementa un sistema basado en acciones ROS 2 utilizando Python, para ello deberás crear un nuevo paquete llamado battery_act, puedes encontrar la info necesaria en el siguiente enlace y en el siguiente. Esta acción se va a encargar de simular un proceso de descarga de la batería de un robot, enviando información sobre el progreso y devolviendo un resultado al finalizar. El objetivo es establecer un valor de batería en el cual el robot debe enviar un aviso de "batería baja" para proceder a su carga. El funcionamiento de la acción sería el siguiente:**

   - **El nodo Servidor llamado battery_charger, recibirá el Goal que será el valor de batería, en tanto porcentual, en el cual el robot deberá mandar el aviso (Ej: 20%). Por tanto, el robot partirá de un 1005 de batería y esta se irá reduciendo**

en un 5% por cada segundo que pase. El Servidor deberá ir actualizando el valor del Feedback mostrando el valor actual de la batería del robot. Asimismo, el Servidor devolverá como Result un aviso (Ej: "Batería Baja, por favor cargue el robot!"). Ten en cuenta que la acción debe poder cancelar si se manda la petición de cancelación.

- **El nodo Cliente llamado battery_client, enviará el Goal al servidor teniendo en cuenta que este valor se recogerá desde la línea de comandos al lanzar el nodo. El nodo Cliente deberá ir publicando el Feedback de la acción, así como el mensaje del Result una vez haya finalizado la acción.**

Creamos el nuevo paquete y dentro de la interfaz, creamos la nueva acción *Battery.action*. Después, podemos crear el servidor y el cliente en el nuevo paquete *battery_act*, creando en el cliente la posibilidad de establecer el límite de batería. Si ejecutamos, tenemos:

```
davidetsm-docker@david:/workspace/ros2_ws$ ros2 run battery_act server
[INFO] [1772020102.702581310] [battery_charger]: Servidor de batería iniciado...
[INFO] [1772020105.280274749] [battery_charger]: Recibido objetivo de aviso: 20%
[INFO] [1772020105.283580466] [battery_charger]: Iniciando descarga de batería...
[INFO] [1772020105.284779730] [battery_charger]: Batería actual: 100%
[INFO] [1772020106.286772142] [battery_charger]: Batería actual: 95%
[INFO] [1772020107.288871556] [battery_charger]: Batería actual: 90%
[INFO] [1772020108.291521009] [battery_charger]: Batería actual: 85%
[INFO] [1772020109.294133490] [battery_charger]: Batería actual: 80%
[INFO] [1772020110.296641517] [battery_charger]: Batería actual: 75%
[INFO] [1772020111.299516036] [battery_charger]: Batería actual: 70%
[INFO] [1772020112.302160675] [battery_charger]: Batería actual: 65%
[INFO] [1772020113.305044449] [battery_charger]: Batería actual: 60%
[INFO] [1772020114.308013317] [battery_charger]: Batería actual: 55%
[INFO] [1772020115.310940208] [battery_charger]: Batería actual: 50%
[INFO] [1772020116.312781944] [battery_charger]: Batería actual: 45%
[INFO] [1772020117.315606148] [battery_charger]: Batería actual: 40%
[INFO] [1772020118.318226055] [battery_charger]: Batería actual: 35%
[INFO] [1772020119.319774435] [battery_charger]: Batería actual: 30%
[INFO] [1772020120.322545822] [battery_charger]: Batería actual: 25%
```

Figure 40: Ejecución de parte del servidor

```
davidetsm-docker@david:/workspace$ ros2 run battery_act client 20
[INFO] [1772020105.322447649] [battery_client]: Feedback: Batería al 100%
[INFO] [1772020106.288629852] [battery_client]: Feedback: Batería al 95%
[INFO] [1772020107.290733214] [battery_client]: Feedback: Batería al 90%
[INFO] [1772020108.293362086] [battery_client]: Feedback: Batería al 85%
[INFO] [1772020109.295934174] [battery_client]: Feedback: Batería al 80%
[INFO] [1772020110.298459144] [battery_client]: Feedback: Batería al 75%
[INFO] [1772020111.301366987] [battery_client]: Feedback: Batería al 70%
[INFO] [1772020112.303967708] [battery_client]: Feedback: Batería al 65%
[INFO] [1772020113.306871892] [battery_client]: Feedback: Batería al 60%
[INFO] [1772020114.310058628] [battery_client]: Feedback: Batería al 55%
[INFO] [1772020115.312819482] [battery_client]: Feedback: Batería al 50%
[INFO] [1772020116.314650841] [battery_client]: Feedback: Batería al 45%
[INFO] [1772020117.317433140] [battery_client]: Feedback: Batería al 40%
[INFO] [1772020118.320103779] [battery_client]: Feedback: Batería al 35%
[INFO] [1772020119.321594819] [battery_client]: Feedback: Batería al 30%
[INFO] [1772020120.324345554] [battery_client]: Feedback: Batería al 25%
[INFO] [1772020121.327856286] [battery_client]: Resultado: ¡Batería Baja, por favor cargue el robot!
```

Figure 41: Ejecución de parte del cliente